

Montañas triples

La Cordillera Oriental en los Andes abarca toda Bolivia. Está formada por una secuencia de N montañas, numeradas de 0 a $N - 1$. La **altura** de la montaña i ($0 \leq i < N$) es $H[i]$, que es un entero entre 1 y $N - 1$, inclusive.

Para cualquier pareja de montañas i y j , donde $0 \leq i < j < N$, la **distancia** entre ellas está definida como $d(i, j) = j - i$.

De acuerdo a antiguas leyendas Inca, una terna de montañas es **mítica** si posee la siguiente propiedad especial: las alturas de las tres montañas **concuerdan** con sus distancias por pares **ignorando el orden**.

Formalmente, una terna de índices (i, j, k) es mítica si

- $0 \leq i < j < k < N$, y
- las alturas $(H[i], H[j], H[k])$ concuerdan con las distancias por pares $(d(i, j), d(i, k), d(j, k))$ ignorando el orden. Por ejemplo, para los índices 0, 1, 2 las distancias de pares son (1, 2, 1), por tanto, las alturas $(H[0], H[1], H[2]) = (1, 1, 2)$, $(H[0], H[1], H[2]) = (1, 2, 1)$, y $(H[0], H[1], H[2]) = (2, 1, 1)$ concuerdan con las distancias, pero las alturas $(H[0], H[1], H[2]) = (1, 2, 2)$ no concuerdan.

Este problema consiste de dos partes, donde cada subtarea es asociada con **Parte I** o **Parte II**. Puedes resolver las subtareas en cualquier orden. En particular, **no se requiere** completar toda la Parte I antes de intentar la Parte II.

Parte I

Dada la descripción de la cordillera de montañas, tu tarea es contar el número de ternas míticas.

Detalles de Implementación

Deberás implementar la siguiente función:

```
long long count_triples(std::vector<int> H)
```

- H : arreglo de largo N , representando las alturas de las montañas.
- Esta función es llamada exactamente una vez para cada caso de prueba.

El procedimiento debe retornar un entero T , el número de ternas míticas en la cordillera.

Restricciones

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$ para cada i tal que $0 \leq i < N$.

Subtareas

La Parte I vale un total de 70 puntos.

Subtarea	Puntuación	Restricciones adicionales
1	8	$N \leq 100$
2	6	$H[i] \leq 10$ para cada i tal que $0 \leq i < N$.
3	10	$N \leq 2000$
4	11	Las alturas son no decrecientes. Es decir, $H[i - 1] \leq H[i]$ para cada i tal que $1 \leq i < N$.
5	16	$N \leq 50\,000$
6	19	Sin restricciones adicionales.

Ejemplo

Considera la siguiente llamada.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Existen 3 ternas míticas en la cordillera:

- Para $(i, j, k) = (1, 3, 4)$, las alturas $(1, 3, 2)$ concuerdan con la distancia a pares $(2, 3, 1)$.
- Para $(i, j, k) = (2, 3, 6)$, las alturas $(4, 3, 1)$ concuerdan con la distancia a pares $(1, 4, 3)$.
- Para $(i, j, k) = (3, 4, 6)$, las alturas $(3, 2, 1)$ concuerdan con la distancia a pares $(1, 3, 2)$.

Por lo tanto, la función debe retornar 3.

Nota que los índices $(0, 2, 4)$ no forman una terna triple, ya que las alturas $(4, 4, 2)$ no concuerdan con las distancias a pares $(2, 4, 2)$.

Parte II

Tu tarea es construir una cordillera con muchas ternas míticas. Esta parte consiste de 6 subtareas de **sólo salida** con **calificación parcial**.

En cada subtarea, te dan dos enteros positivos M y K , y debes construir una cordillera con **a lo sumo** M montañas. Si tu solución contiene **al menos** K ternas míticas, recibirás puntuación completa para esa subtarea. De lo contrario, tu puntuación será proporcional al número de ternas míticas que contenga tu solución.

Nota que tu solución debe contener una cordillera válida. Específicamente, supongamos que tu solución tiene N montañas (N debe satisfacer $3 \leq N \leq M$). Entonces, la altura de la montaña i ($0 \leq i < N$), denotada por $H[i]$, debe contener un entero entre 1 y $N - 1$, inclusivo.

Detalles de implementación

Hay dos métodos para presentar tu solución, y puedes usar cualquiera de las dos para cada subtarea.

- **Archivo de salida**
- **Llamada a función**

Para enviar tu solución a través de un **archivo de salida**, crea y envía un archivo de texto con el siguiente formato:

```
N
H[0] H[1] ... H[N-1]
```

Para enviar tu solución a través de una **llamada de función**, debes implementar la siguiente función:

```
std::vector<int> construct_range(int M, int K)
```

- M : el máximo número de montañas.
- K : el número deseado de ternas míticas.
- Esta función es llamada exactamente una vez por cada subtarea.

La función debe retornar un arreglo H de largo N , representando las alturas de las montañas.

Subtareas y Puntuación

La Parte II vale un total de 30 puntos. Para cada subtarea, los valores de M y K son fijos y mostrados en la siguiente tabla:

Subtarea	Puntuación	M	K
7	5	20	30
8	5	500	2000
9	5	5000	50 000
10	5	30 000	700 000
11	5	100 000	2 000 000
12	5	200 000	12 000 000

Para cada subtarea, si tu solución no forma una cordillera válida, tu puntuación será 0 (reportada como `Output isn't correct` en el CMS).

De lo contrario, sea T el número de ternas míticas en tu solución.

Entonces, tu puntuación para la subtarea es:

$$5 \cdot \min \left(1, \frac{T}{K} \right).$$

Calificador local

Las Partes I y II usan el mismo calificador local, con la distinción entre las dos partes determinada por la primera línea de entrada.

Formato de entrada para la Parte I:

```
1
N
H[0] H[1] ... H[N-1]
```

El formato de salida para la Parte I:

```
T
```

Formato de entrada para la Parte II:

```
2
M K
```

Formato de salida para la Parte II:

N

H[0] H[1] ... H[N-1]

Nota que la salida del calificador local concuerda con el formato requerido para el archivo de salida para la Parte II.